

RevenueGuard

Summary, technical statement and workflow.

A short read: the problem in a paragraph, the technical statement in one sentence, the seven-stage control loop, how the recovery figure is measured, and the results.

Rs 85.63 L

recovered per 2 days (mean of 8 seeds)

26.6%

of money the incidents put at risk

0/8

seeds where the router lost money

The problem

Payment failures are almost never global. They are narrow — one gateway, one method, one issuing bank — and a narrow failure is invisible on the chart everyone watches. A slice carrying 4% of volume can collapse from 96% to 40% and move the headline success rate by about two points, well inside normal daily variation. Nobody pages, and the money leaves quietly until a human spots a pattern in a support queue.

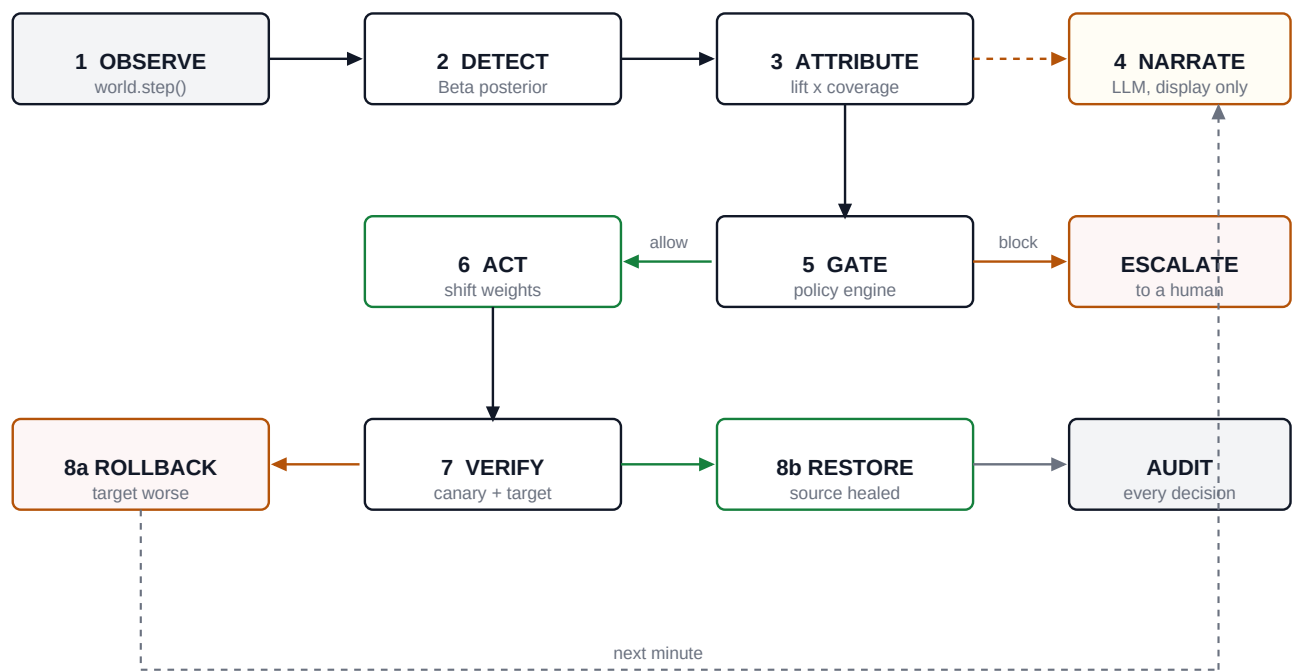
The technical statement

Detect per-slice success-rate degradation under two hostile statistical conditions — very low volume and very high test multiplicity — attribute the alarming population to a common dimension deterministically, and execute a bounded, reversible, fully audited traffic shift whose recovered revenue is measured against a control arm rather than projected.

Each clause of that sentence is a design constraint, and each was chosen against a more obvious alternative:

Constraint	Obvious approach	What is done, and why
Ground truth	Label synthetic rows, train a classifier, report precision and recall.	Outages are <i>scheduled</i> , so time-to-detect is latency against an event we caused. Training on self-generated labels measures the generator, not the world.
Detection	Threshold or z-score per slice.	Beta posterior with cohort shrinkage. A slice with four attempts a minute swings 50–100% on noise; 207,360 tests over two days drown any fixed significance level.
Attribution	Ask an LLM what is wrong.	Lift with coverage, deterministic. A sampled token in the path of a money-moving action cannot be reproduced or defended.
Action	Reroute on detection.	A separate policy engine decides <i>permission</i> . Ten bounds, stopping rules, and every refusal recorded with its rule.
Reversal	Drain the bad gateway to zero.	A 3% canary floor. A gateway with no traffic emits no observations, so you could never tell it recovered or route back.
Measurement	Count recoveries, multiply by ticket size.	Paired control and treatment on bit-identical demand, repeated across seeds, reported as a confidence interval.

The workflow

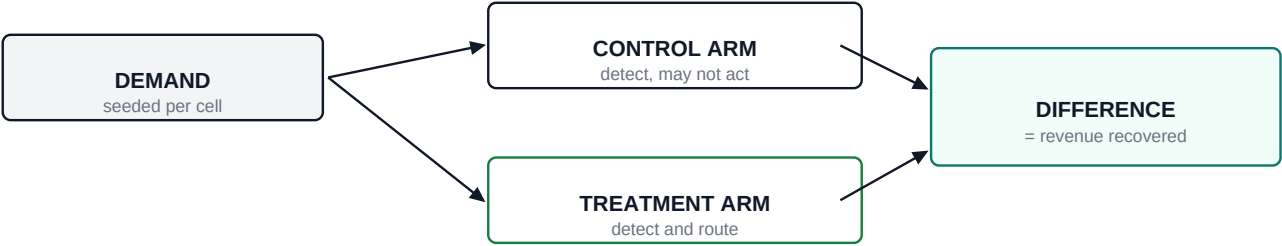


Seven stages, once a minute. The system sees only (slice, minute, attempts, successes) — never the incident plan that generated the traffic.

Stage	Guarantee it provides
1 Observe	No ground truth is visible to any decision-making code.
2 Detect	Alarms only when the drop is both statistically real and materially large.
3 Attribute	Deterministic and reproducible; refuses to name a cause the evidence cannot support.
4 Narrate	Display only. Cannot reach the ledger or any decision.
5 Gate	Ten explicit bounds; every verdict recorded with its rule, including refusals.
6 Act	Bounded fraction, per key, with a canary always left behind.
7 Verify	Rolls back if the destination proved worse; restores in thirds once the source heals.

How the measurement works

One seed, one incident plan, one demand stream



Guard: total attempts must match exactly across arms.
If they differ the run refuses to report a recovery figure at all - the comparison would be invalid.
Repeated across seeds; the headline is the confidence interval, not any single run.

Where AI is used

DETERMINISTIC
statistics, rules, arithmetic

detection - attribution - policy - routing - rollback - audit
Reproducible. Same input, same decision, forever.

LANGUAGE MODEL
prose only

restates a finished attribution for the operator console
Never decides. Never reaches the ledger. Falls back to a template.

Why the line sits here

A sampled token in the path of a money-moving action cannot be reviewed, reproduced, or defended.
Two tests enforce the boundary: the prompt carries no raw traffic, and no decision-path module imports the narrator.

Results

Rs 85.63 L
recovered per 2 days, mean of 8 paired seeds

26.6%
of the money the incidents put at risk

0/8
seeds where the router lost money

Metric	Mean	SD	Min	Max
Revenue recovered	Rs 85.63 L	Rs 4.00 L	Rs 79.89 L	Rs 91.02 L
Share of exposure	26.6%	1.2%	24.7%	28.2%
Success rate gain	0.394 pp	0.019	0.368	0.425
Routing actions	127.2	6.7	113	135
Rollbacks	29.4	2.7	26	35

95% CI on mean recovery: **Rs 82.29 L to Rs 88.97 L**, which excludes zero. Cross-checked independently: measured recovery Rs 87.33 L against an exposure reduction of Rs 86.03 L — different quantities, agreeing within 1.5%.

What is deliberately not claimed

- **The world is simulated.** Gateway health, demand and incidents are ours; the detector's view of them is the part under evaluation.
- **Gateway health does not degrade under load.** A real acquirer would push back as traffic arrives, so the recovery figure is an upper bound. This is the first item on the roadmap.
- **Routing weights cannot be set through Razorpay's public API.** Acquirer selection is Razorpay's own product. The executor demonstrates the half that is real — test-mode Orders carrying the decision context in their notes.
- **27% of exposure, not 90%.** Detection latency and deliberate policy caps set the ceiling.
- **About 29 rollbacks per 127 actions** are reported rather than tuned away.

Next

#	Item	Why it matters
1	Load-dependent gateway health	The one missing dynamic that could change a conclusion. Today the router can shift traffic onto a destination without degrading it; a real acquirer would push back. Until this exists the recovery figure is an upper bound.
2	Sensitivity analysis over the policy configuration	Ten bounds are currently defended by argument, not measurement. A sweep over each, reporting recovery and rollback rate, would show which earn their place and which are superstition.
3	Cost of action	Rerouting is not free: acquirer contracts carry volume commitments and per-transaction pricing differences. Netting that against recovered revenue turns a gross figure into a net one.
4	Detector ensemble	The benchmark already shows each detector catches incidents the other misses. Running both and taking the union, evaluated at a matched false-alarm rate, is the obvious next gain and is cheap.

Full detail in the companion document, *RevenueGuard — Complete Project Documentation*.